OPERATING SYSTEMS LABORATORY RECORD

Student: Brinda C V **Roll No:** 241701011
Department: CSD (Computer Science & Design) **Academic Year:** 2025 - 2026 | Year: 3rd Year

EX NO: 1 — Basic Linux Commands (General Purpose)

AIM:

> To execute and understand basic Linux general purpose utility commands.

1.1 GENERAL PURPOSE COMMANDS:

Command	Description & Syntax	Example & Options
`date`	Displays current system date & time. **Syntax:** `$ date`	`date +%m` (Month), `date +%d` (Day), `date +%y` (Year), `date +%H:%M:%S`
`echo`	Prints standard text string. **Syntax:** `$ echo [string]`	`$ echo "God is Great"`
`cal`	Displays calendar month/year. **Syntax:** `$ cal [month] [year]`	`$ cal Jan 2012`
`bc`	Command-line arbitrary precision calculator. **Syntax:** `$ bc`	`$ bc -l` → `16/4` → Output: `4`
`who`	Shows active logged in users. **Syntax:** `$ who`	`$ who`
`who am i`	Displays terminal user session login info. **Syntax:** `$ who am i`	`$ who am i`
`id`	Displays numeric User ID (UID) & Group ID (GID). **Syntax:** `$ id`	`$ id`
`tty`	Prints current terminal device filename. **Syntax:** `$ tty`	`$ tty`
`clear`	Clears active terminal screen. **Syntax:** `$ clear`	`$ clear`
`man`	Opens reference manual for command. **Syntax:** `$ man [command]`	`$ man ls`
`ps`	Reports dynamic process snapshot. **Syntax:** `$ ps`	`$ ps -e`, `$ ps -aux`
`uname`	Prints system OS and hardware details. **Syntax:** `$ uname [option]`	`-m` (Machine), `-n` (Node), `-r` (Release), `-a` (All details)

1.2 DIRECTORY HANDLING COMMANDS:

Command	Syntax	Purpose & Example
pwd	$ pwd	Print working directory path.
mkdir	$ mkdir [dirname]	Create empty directory. Example: $ mkdir receee
rmdir	$ rmdir [dirname]	Remove an empty directory. Example: $ rmdir receee
cd	$ cd [dirname]	Change current working directory path. Example: $ cd receee
ls	$ ls [options]	List directory contents. Options: ls -l (long format), ls -a (all including hidden)

1.3 FILE HANDLING COMMANDS:

Command	Syntax	Purpose & Examples
cat	$ cat > filename $ cat filename	Create file using input redirection. View file contents on terminal standard output.
cp	$ cp oldfile newfile	Copy contents of oldfile to newfile.
rm	$ rm [options] filename	Remove files/directories. Options: rm -f (force), rm -fr (recursive directory delete)
mv	$ mv oldfile newfile	Move/rename files or directory paths.
file	$ file filename	Determine file storage structure/type classification.
wc	$ wc filename	Count lines, words, and byte/character size in file.
> (Redirection)	$ command > filename	Redirect standard output stream into designated file.
| (Pipes)	$ cmd1 | cmd2	Connects stdout of cmd1 directly to stdin of cmd2.
tee	$ cmd | tee filename	Splits standard output into both screen and saved file.

METACHARACTERS & FILE PERMISSIONS:

Metacharacters: * (matches zero/more chars), ? (matches single char), [...] (character range set), ! (NOT logical matching).
Examples: ls r* (files starting with 'r'), ls ?kkk (matches any 1st char followed by 'kkk'), ls [a-m] (files starting with letters a to m).
File Permissions Structure: 10-char string e.g., -rwxr-xr-- (User: rwx, Group: r-x, Others: r--).
Permissions Modifier (chmod):

$ chmod u-wx college # Removes write & execute permission from owner $ chmod u+rw,g+rw college # Adds read & write permission for user and group $ chmod 761 college # Octal: 7(rwx) for owner, 6(rw-) for group, 1(--x) for others

1.4 GROUPING OPERATORS:

- `;` (Semicolon): Executes commands sequentially regardless of success/failure. (e.g., `$ who; date`)
- `&&` (Logical AND): Executes subsequent command ONLY if previous command succeeds (exit status 0). (e.g., `$ who && date`)
- `||` (Logical OR): Executes subsequent command ONLY if preceding command fails (non-zero status). (e.g., `$ who || date`)

1.5 UNIX FILTERS & TEXT PROCESSING:

Filter	Syntax	Description
`head`	`$ head -n filename`	Displays top n lines of file (default 10). Ex: `head -5 college`
`tail`	`$ tail -n filename`	Displays bottom n lines of file (default 10). Ex: `tail -5 college`
`more / pg`	`$ command | more`	Displays lengthy file output screen by screen.
`grep`	`$ grep "pattern" file`	Searches for string patterns using regular expression.
`sort`	`$ sort [options] file`	Sorts file lines alphabetically/numerically. Options: `-u` (unique), `-r` (reverse), `-n` (numeric)
`nl`	`$ nl filename`	Numbers text lines of specified file for display.
`cut`	`$ cut -c [pos] file`	Extracts specified character columns from file lines.

1.6 OTHER ESSENTIAL SYSTEM ADMINISTRATION COMMANDS:

1. free: Displays memory utilization (RAM & Swap).

```
[root@localhost ~]# free -t total used free shared buff/cache available Mem: 4044380 605464 2045080 148820 1393836 3226708 Swap: 2621436 0 2621436 Total: 6665816 605464 4666516
```

2. top: Real-time dynamic system task viewer.

```
[root@localhost ~]# top top - 08:07:28 up 24 min, 2 users, load average: 0.01, 0.06, 0.23 Tasks: 211 total, 1 running, 210 sl eeping, 0 stopped, 0 zombie %Cpu(s): 0.8 us, 0.3 sy, 0.0 ni, 98.9 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
```

3. df: Disk filesystem resource usage statistics.

```
[root@localhost ~]# df -h Filesystem Size Used Avail Use% Mounted on /dev/sda6 487M 168M 290M 37% /boot
```

4. ping & traceroute: Network connectivity and packet hop path diagnostic.

```
[root@localhost ~]# ping -c 2 172.16.4.1 64 bytes from 172.16.4.1: icmp_seq=1 ttl=64 time=0.328 ms [root@localhost ~]# traceroute www.rajalakshmi.org 1 gateway (172.16.4.1) 0.299 ms 0.297 ms
```

EX NO: 2A — Shell Script: Basic Calculator

> To write a Shell Script to implement a basic calculator supporting arithmetic operations.

PROGRAM (ARITH.SH):

```
echo "enter two no" read a read b c=`expr $a + $b` d=`expr $a - $b` e=`expr $a \* $b` f=`expr $a / $b` g=`expr $a % $b` echo "add $c" echo "sub $d" echo "mul $e" echo "div $f" echo "mod $g"
```

OUTPUT:

```
[REC@localhost ~]$ sh arith.sh enter two no 5 10 add 15 sub -5 mul 50 div 0 mod 5
```

EX NO: 2B — Shell Script: Leap Year Verification

> To write a Shell Script to test whether a given year is a leap year using conditional statements.

PROGRAM (LEAP.SH):

```
echo "enter number" read a m=`expr $a % 4` if [ $m -eq 0 ] then echo "leap year" else echo "not leap year" fi
```

OUTPUT:

```
[REC@localhost ~]$ sh leap.sh enter number 12 leap year
```

EX NO: 3A — Shell Script: Reverse of a Digit

> To write a Shell Script to reverse a given integer number using looping construct.

PROGRAM (REVERSE.SH):

```
echo "enter number" read n rev=0 while [ $n -gt 0 ] do d=`expr $n % 10` rev=$(echo "$rev * 10 + $d" | bc) n=`expr $n / 10` done echo "Reversed number: $rev"
```

OUTPUT:

```
[REC@localhost ~]$ sh reverse.sh enter number 123 Reversed number: 321
```

EX NO: 3B — Shell Script: Fibonacci Series Generation

> To write a Shell Script to generate Fibonacci series up to N terms using a for loop.

PROGRAM (FIBONACCI.SH):

```
echo "enter number" read n a=-1 b=1 echo "fibonacci series" for((i=1;i<=$n;i++)) do c=$(echo "$a + $b" | bc) echo $c a=$b b=$c done
```

OUTPUT:

```
[REC@localhost ~]$ sh fibonacci.sh enter number 7 fibonacci series 0 1 1 2 3 5 8
```

EX NO: 4A — AWK Script: Employee Average Pay Calculation

> To find out the average pay of all employees whose salary per day is > 6000 and days worked > 4 using AWK script.

ALGORITHM:

1. Create flat data file `emp.dat` (Name, Salary/day, Days worked).
2. Create AWK script `emp.awk`.
3. For each record, test condition: `$2 > 6000 && $3 > 4`.
4. If true, print name & earned pay (`$2 * $3`), accumulate total pay and increment matching count.
5. In `END` block, compute and display count, total pay, and average salary.

DATA FILE (EMP.DAT) & SCRIPT (EMP.AWK):

```
# Input Data: emp.dat JOE 8000 5 RAM 6000 5 TIM 5000 6 BEN 7000 7 AMY 6500 6 # Script Code: emp.awk BEG IN { print "EMPLOYEES DETAILS" } { if($2 > 6000 && $3 > 4) { print $1, "\t\t", $2*$3 pay = pay + $2*$3 count = count + 1 } } END { print "no of employees are=", count print "total pay=", pay print "average pay=", pay/count }
```

OUTPUT:

```
[student@localhost ~]$ gawk -f emp.awk emp.dat EMPLOYEES DETAILS JOE 40000 BEN 49000 AMY 39000 no o f employees are= 3 total pay= 128000 average pay= 42666.7
```

EX NO: 4B — AWK Script: Examination Results Evaluation

> To process student mark sheets and evaluate Pass/Fail status based on subject marks threshold (<45 fails).

DATA FILE (MARKS.DAT) & SCRIPT CODE (MARKS.AWK):

```
# File: marks.dat BEN 40 55 66 77 55 77 TOM 60 67 84 92 90 60 RAM 90 95 84 87 56 70 JIM 60 70 65 78 90 87 # Script: marks.awk BEGIN { print "NAME\tSUB-1\tSUB-2\tSUB-3\tSUB-4\tSUB-5\tSUB-6\tSTATUS\n" } { if ($2 < 45 || $3 < 45 || $4 < 45 || $5 < 45 || $6 < 45 || $7 < 45) { print $1,"\t",$2,"\t",$3,"\t",$4,"\t",$5,"\t",$6,"\t",$7,"\t","FAIL" } else { print $1,"\t",$2,"\t",$3,"\t",$4,"\t",$5,"\t",$6,"\t",$7,"\t","PASS" } } END { print "\n" }
```

OUTPUT:

```
[root@localhost student]# gawk -f marks.awk marks.dat NAME SUB-1 SUB-2 SUB-3 SUB-4 SUB-5 SUB-6 STATUS BEN 40 55 66 77 55 77 FAIL TOM 60 67 84 92 90 60 PASS RAM 90 95 84 87 56 70 PASS JIM 60 70 65 78 90 87 PAS S
```

EX NO: 5 — System Calls Programming (fork & process IDs)

> To experiment process creation system calls using fork(), getpid(), and getppid() functions in C.

PROGRAM CODE (FORK_DEMO.C):

```
#include <stdio.h> #include <stdlib.h> #include <unistd.h> int main() { int pid; pid = fork(); printf("\n THIS LINE EX ECUTED TWICE"); if(pid == -1) { printf("\n CHILD PROCESS NOT CREATED\n"); exit(0); } if(pid == 0) { printf("\n I AM C HILD PROCESS AND MY ID IS %d \n", getpid()); printf("\n THE CHILD PARENT PROCESS ID IS:%d \n", getppid()); } else { printf("\n I AM PARENT PROCESS AND MY ID IS:%d\n", getpid()); printf("\n THE PARENTS PARENT PROCESS ID IS:% d\n", getppid()); } printf("\n IT CAN BE EXECUTED TWICE\n"); return 0; }
```

OUTPUT:

```
[student@localhost ~]$ gcc fork_demo.c -o fork_demo [student@localhost ~]$ ./fork_demo THIS LINE EXECUTE D TWICE I AM PARENT PROCESS AND MY ID IS:3421 THE PARENTS PARENT PROCESS ID IS:2104 IT CAN BE EXECU TED TWICE THIS LINE EXECUTED TWICE I AM CHILD PROCESS AND MY ID IS 3422 THE CHILD PARENT PROCESS I D IS:3421 IT CAN BE EXECUTED TWICE
```

EX NO: 6A — CPU Scheduling: First Come First Serve (FCFS)

To write a program to simulate First-Come First-Serve (FCFS) CPU scheduling algorithm.

PROGRAM CODE (FCFS.PY):

```
print("Enter the number of process: ") n = int(input()) print("Enter the burst time of the processes: ") bt = list(map(i nt, input().split()))) wt, tat = [0], [bt[0]] avgwt, avgtat = 0, 0 for i in range(1, n): wt.append(wt[i-1] + bt[i-1]) tat.append(wt [i] + bt[i]) avgwt = sum(wt) / float(n) avgtat = sum(tat) / float(n) print("\nProcess\t Burst Time\t Waiting Time\t Turnar ound Time") for i in range(n): print(f"{i}\t\t{bt[i]}\t\t{wt[i]}\t\t{tat[i]}") print(f"\nAverage Waiting time is: {avgwt}") print (f"Average Turn Around Time is: {avgtat}")
```

OUTPUT:

```
Enter the number of process: 3 Enter the burst time of the processes: 24 3 3 Process Burst Time Waiting Time Tu rnaround Time 0 24 0 24 1 3 24 27 2 3 27 30 Average Waiting time is: 17.0 Average Turn Around Time is: 19.0
```

EX NO: 6B — CPU Scheduling: Shortest Job First (SJF)

To implement Shortest Job First (SJF) non-preemptive CPU scheduling algorithm.

PROGRAM CODE (SJF.PY):

```
n = int(input("Enter the number of process: ")) processes = list(range(1, n + 1)) bt = list(map(int, input("Enter the b urst time of the processes: ").split())) # Bubble sort processes according to Burst Time for i in range(n - 1): for j in ran ge(n - i - 1): if bt[j] > bt[j+1]: bt[j], bt[j+1] = bt[j+1], bt[j] processes[j], processes[j+1] = processes[j+1], processes[j] wt, tat = [0], [bt[0]] for i in range(1, n): wt.append(wt[i-1] + bt[i-1]) tat.append(wt[i] + bt[i]) print("\nProcess\t Burst Time\t Wait ing Time\t Turnaround Time") for i in range(n): print(f"{processes[i]}\t\t{bt[i]}\t\t{wt[i]}\t\t{tat[i]}") print(f"Average Waiti ng time is: {sum(wt)/float(n)}") print(f"Average Turn Around Time is: {sum(tat)/float(n)}")
```

OUTPUT:

```
Enter the number of process: 4 Enter the burst time of the processes: 8 4 9 5 Process Burst Time Waiting Time Tu rn Around Time 2 4 0 4 4 5 4 9 1 8 9 17 3 9 17 26 Average Waiting time is: 7.5 Average Turn Around Time is: 13.0
```

EX NO: 6C — CPU Scheduling: Priority Scheduling

To implement Priority CPU scheduling algorithm based on assigned process priority levels.

PROGRAM CODE (PRIORITY.C):

```
#include <stdio.h> int main() { int bt[20], p[20], wt[20], tat[20], pr[20], i, j, n, total=0, pos, temp; float avg_wt, avg_tat; printf("Enter Total Number of Process: "); scanf("%d", &n); printf("Enter Burst Time and Priority\n"); for(i=0; i<n; i++) { printf("\nP[%d]\n", i+1); printf("Burst Time: "); scanf("%d", &bt[i]); printf("Priority: "); scanf("%d", &pr[i]); p[i] = i + 1; } // Sort according to Priority (Ascending) for(i=0; i<n; i++) { pos = i; for(j=i+1; j<n; j++) { if(pr[j] < pr[pos]) pos = j; } temp=pr[i]; pr[i]=pr[pos]; pr[pos]=temp; temp=bt[i]; bt[i]=bt[pos]; bt[pos]=temp; temp=p[i]; p[i]=p[pos]; p[pos]=temp; } wt[0] = 0; for(i=1; i<n; i++) { wt[i] = 0; for(j=0; j<i; j++) wt[i] += bt[j]; total += wt[i]; } avg_wt = (float)total / n; total = 0; printf("\nProcess \t Burst Time \tWaiting Time\tTurnaround Time"); for(i=0; i<n; i++) { tat[i] = bt[i] + wt[i]; total += tat[i]; printf("\nP[%d]\t \t %d\t\t %d\t\t\t%d", p[i], bt[i], wt[i], tat[i]); } avg_tat = (float)total / n; printf("\n\nAverage Waiting Time = %.2f", avg_wt); printf("\nAverage Turnaround Time = %.2f\n", avg_tat); return 0; }
```

OUTPUT:

```
Enter Total Number of Process: 3 P[1] Burst Time: 10, Priority: 2 P[2] Burst Time: 1, Priority: 1 P[3] Burst Time: 2, Priority: 3 Process Burst Time Waiting Time Turnaround Time P[2] 1 0 1 P[1] 10 1 11 P[3] 2 11 13 Average Waiting Time = 4.00 Average Turnaround Time = 8.33
```

EX NO: 6D — CPU Scheduling: Round Robin (RR)

To implement Round Robin (RR) preemptive CPU scheduling algorithm using predefined time quantum.

PROGRAM CODE (ROUND_ROBIN.C):

```
#include <stdio.h> int main() { int i, limit, total = 0, x, counter = 0, time_quantum; int wait_time = 0, turnaround_time = 0, arrival_time[10], burst_time[10], temp[10]; float average_wait_time, average_turnaround_time; printf("Enter Total Number of Processes: "); scanf("%d", &limit); x = limit; for(i = 0; i < limit; i++) { printf("Details of Process[%d] - Arrival & Burst Time: ", i + 1); scanf("%d %d", &arrival_time[i], &burst_time[i]); temp[i] = burst_time[i]; } printf("Enter Time Quantum: "); scanf("%d", &time_quantum); printf("\nProcess ID\tBurst Time\tTurnaround Time\tWaiting Time\n"); for(total = 0, i = 0; x != 0;) { if(temp[i] <= time_quantum && temp[i] > 0) { total = total + temp[i]; temp[i] = 0; counter = 1; } else if(temp[i] > 0) { temp[i] = temp[i] - time_quantum; total = total + time_quantum; } if(temp[i] == 0 && counter == 1) { x--; printf("Process[%d]\t\t%d\t\t%d\t\t%d\n", i + 1, burst_time[i], total - arrival_time[i], total - arrival_time[i] - burst_time[i]); wait_time += total - arrival_time[i] - burst_time[i]; turnaround_time += total - arrival_time[i]; counter = 0; } if(i == limit - 1) i = 0; else if(arrival_time[i + 1] <= total) i++; else i = 0; } printf("\nAverage Waiting Time:\t%f", wait_time * 1.0 / limit); printf("\nAvg Turnaround Time:\t%f\n", turnaround_time * 1.0 / limit); return 0; }
```

OUTPUT:

```
Enter Total Number of Processes: 3 Process[1] Arrival: 0 Burst: 24 Process[2] Arrival: 0 Burst: 3 Process[3] Arrival: 0 Burst: 3 Enter Time Quantum: 4 Process ID Burst Time Turnaround Time Waiting Time Process[2] 3 7 4 Process[3] 3 10 7 Process[1] 24 30 6 Average Waiting Time: 5.666667 Avg Turnaround Time: 15.666667
```

EX NO: 7 — Inter-Process Communication Using Shared Memory

To implement Inter-Process Communication (IPC) using Shared Memory functions (shmget, shmat, shmdt).

SENDER PROGRAM (SENDER.C) & RECEIVER PROGRAM (RECEIVER.C):

```
/* sender.c */ #include <sys/ipc.h> #include <sys/shm.h> #include <stdio.h> #include <stdlib.h> #include <unistd.h> #define SharedMemSize 50 void main() { int shmid; key_t key = 5677; char *shared_memory; if ((shmid = shmget(key, SharedMemSize, IPC_CREAT | 0666)) < 0) { perror("shmget"); exit(1); } if((shared_memory = shmat(shmid, NULL, 0)) == (char *) -1) { perror("shmat"); exit(1); } sprintf(shared_memory, " Welcome to Shared Memory"); sleep(2); exit(0); } /* receiver.c */ #include <sys/ipc.h> #include <sys/shm.h> #include <stdio.h> #include <stdlib.h> #define SharedMemSize 50 void main() { int shmid; key_t key = 5677; char *shared_memory; if ((shmid = shmget(key, SharedMemSize, 0666)) < 0) { perror("shmget"); exit(1); } if((shared_memory = shmat(shmid, NULL, 0)) == (char *) -1) { perror("shmat"); exit(1); } printf("Message Received: %s \n", shared_memory); exit(0); }
```

OUTPUT EXECUTION ACROSS TERMINALS:

```
Terminal 1: [root@localhost student]# gcc sender.c -o sender [root@localhost student]# ./sender Terminal 2: [root@localhost student]# gcc receiver.c -o receiver [root@localhost student]# ./receiver Message Received: Welcome to Shared Memory
```

EX NO: 8 — Producer-Consumer Problem Using Semaphores

To write a C program to solve classical Producer-Consumer synchronization problem using mutex and counting semaphores.

PROGRAM CODE (PRODUCER_CONSUMER.C):

```
#include <stdio.h> #include <stdlib.h> int mutex = 1, full = 0, empty = 3, x = 0; int wait(int s) { return (--s); } int signal(int s) { return (++s); } void producer() { mutex = wait(mutex); full = signal(full); empty = wait(empty); x++; printf("\nProducer produces the item %d", x); mutex = signal(mutex); } void consumer() { mutex = wait(mutex); full = wait(full); empty = signal(empty); printf("\nConsumer consumes item %d", x); x--; mutex = signal(mutex); } int main() { int n; printf("\n1.Producer\n2.Consumer\n3.Exit"); while(1) { printf("\nEnter your choice:"); scanf("%d", &n); switch(n) { case 1: if((mutex==1) && (empty!=0)) producer(); else printf("Buffer is full!!"); break; case 2: if((mutex==1) && (full!=0)) consumer(); else printf("Buffer is empty!!"); break; case 3: exit(0); } } return 0; }
```

OUTPUT EXECUTION TRACE:

```
1.Producer 2.Consumer 3.Exit Enter your choice: 1 Producer produces the item 1 Enter your choice: 2 Consumer consumes item 1 Enter your choice: 2 Buffer is empty!! Enter your choice: 1 Producer produces the item 1 Enter your choice: 1 Producer produces the item 2 Enter your choice: 1 Producer produces the item 3 Enter your choice: 1 Buffer is full!! Enter your choice: 3
```

EX NO: 9 — Deadlock Avoidance (Banker's Algorithm)

> To find out a safe sequence using Banker's algorithm for deadlock avoidance.

ALGORITHM:

1. Initialize `work = available` and `finish[i] = false` for all `i`.
2. Find an `i` such that both `finish[i] == false` and `Need[i] <= work`.
3. If no such `i` exists, go to step 6.
4. Compute `work = work + Allocation[i]`.
5. Assign `finish[i] = true` and go to step 2.
6. If `finish[i] == true` for all `i`, then print the safe sequence.
7. Else, print that there is no safe sequence.

PROGRAM CODE (BANKERS.C):

```
#include <stdio.h> int main() { int n = 5, m = 3, i, j, k; int alloc[5][3] = { {0,1,0}, {2,0,0}, {3,0,2}, {2,1,1}, {0,0,2} }; int max[5][3] = { {7,5,3}, {3,2,2}, {9,0,2}, {2,2,2}, {4,3,3} }; int avail[3] = { 3, 3, 2 }; int f[5] = {0}, ans[5], ind = 0, need[5][3]; for (i = 0; i < n; i++) for (j = 0; j < m; j++) need[i][j] = max[i][j] - alloc[i][j]; for (k = 0; k < 5; k++) { for (i = 0; i < n; i++) { if (f[i] == 0) { int flag = 0; for (j = 0; j < m; j++) { if (need[i][j] > avail[j]) { flag = 1; break; } } if (flag == 0) { ans[ind++] = i; for (int y = 0; y < m; y++) avail[y] += alloc[i][y]; f[i] = 1; } } } printf("The SAFE Sequence is \n"); for (i = 0; i < n - 1; i++) printf(" P%d ->", ans[i]); printf(" P%d\n", ans[n - 1]); return 0; }
```

OUTPUT:

```
The SAFE Sequence is P1 -> P3 -> P4 -> P0 -> P2
```

EX NO: 10A — Memory Allocation: Best Fit

> To implement Best Fit memory allocation technique.

ALGORITHM:

1. Input memory blocks and processes with their respective sizes.
2. Initialize all memory blocks as free.
3. Pick each process and find the smallest block size that is greater than or equal to the process size.
4. If found, allocate the process to that block and reduce the block's available size.
5. If not found, leave the process unallocated and proceed to check the next processes.

PROGRAM CODE (BEST_FIT.C):

```
#include <stdio.h> #include <string.h> void bestFit(int blockSize[], int m, int processSize[], int n) { int allocation[n]; memset(allocation, -1, sizeof(allocation)); for (int i = 0; i < n; i++) { int bestIdx = -1; for (int j = 0; j < m; j++) { if (blockSize[j] >= processSize[i]) { if (bestIdx == -1 || blockSize[bestIdx] > blockSize[j]) bestIdx = j; } } if (bestIdx != -1) { allocation[i] = bestIdx; blockSize[bestIdx] -= processSize[i]; } } printf("\nProcess No. \tProcess Size\tBlock no. \n"); for (int i = 0; i < n; i++) { printf("%d \t\t %d ", i + 1, processSize[i]); if (allocation[i] != -1) printf("\t\t%d\n", allocation[i] + 1); else printf("\t\tNot Allocated\n"); } } int main() { int blockSize[] = {100, 500, 200, 300, 600}; int processSize[] = {212, 417, 112, 426}; int m = sizeof(blockSize) / sizeof(blockSize[0]); int n = sizeof(processSize) / sizeof(processSize[0]); bestFit(blockSize, m, processSize, n); return 0; }
```

OUTPUT:

```
Process No. Process Size Block no. 1 212 4 2 417 2 3 112 3 4 426 5
```

EX NO: 10B — Memory Allocation: First Fit

> To write a C program to implement First Fit memory allocation for fixed partition.

ALGORITHM:

1. Define maximum limit `MAX = 25`.
2. Declare tracking arrays for block sizes, file sizes, flags, and fragments.
3. Read total number of blocks, number of files/processes, and their respective sizes.
4. Iterate through each file and assign it to the first block that has sufficient size (where `block_size >= file_size`).
5. Calculate internal fragmentation and flag allocated blocks. Display allocation table.

PROGRAM CODE (FIRST_FIT.C):

```
#include <stdio.h> #define max 25 void main() { int frag[max], b[max], f[max], i, j, nb, nf, temp; static int bf[max], ff[max]; printf("\nEnter the number of blocks:"); scanf("%d", &nb); printf("Enter the number of files:"); scanf("%d", &nf); printf("\nEnter the size of the blocks:-\n"); for(i = 1; i <= nb; i++) { printf("Block %d:", i); scanf("%d", &b[i]); } printf("Enter the size of the files:-\n"); for(i = 1; i <= nf; i++) { printf("File %d:", i); scanf("%d", &f[i]); } for(i = 1; i <= nf; i++) { for(j = 1; j <= nb; j++) { if(bf[j] != 1) { temp = b[j] - f[i]; if(temp >= 0) { ff[i] = j; break; } } } frag[i] = temp; bf[ff[i]] = 1; } printf("\nFile_no:\tFile_size:\tBlock_no:\tBlock_size:\tFragment"); for(i = 1; i <= nf; i++) printf("\n%d\t\t%d\t\t%d\t\t%d\t\t%d", i, f[i], ff[i], b[ff[i]], frag[i]); }
```

OUTPUT:

```
Enter the number of blocks: 3 Enter the number of files: 2 Enter the size of the blocks:- Block 1: 50 Block 2: 150 Block 3: 300 Enter the size of the files:- File 1: 100 File 2: 40 File_no: File_size: Block_no: Block_size: Fragment 1 100 2 150 50 2 40 1 50 10
```

EX NO: 11A — Page Replacement: FIFO

To find out the number of page faults that occur using First-In First-Out (FIFO) page replacement technique.

PROGRAM CODE (FIFO.PY):

```
def fifo(): global a, n, m f = -1 page_faults = 0 page = [-1] * m for i in range(n): flag = 0 for j in range(m): if page[j] == a[i]: flag = 1 break if flag == 0: f = (f + 1) % m page[f] = a[i] page_faults += 1 print(f"\n{a[i]} -> ", end="") for j in range(m): print(f"{page[j]} " if page[j] != -1 else "- ", end="") else: print(f"\n{a[i]} -> No Page Fault", end="") print(f"\n\nTotal page faults : {page_faults}.") a = [] n = int(input("Enter the size of reference string : ")) for i in range(n): a.append(int(input(f"Enter [{i+1:2d}] : "))) m = int(input("\nEnter page frame size : ")) fifo()
```

OUTPUT EXECUTION:

```
[root@localhost student]# python fifo.py Enter the size of reference string : 20 Reference String: 7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1 Enter page frame size : 3 7 -> 7 - - 0 -> 7 0 - 1 -> 7 0 1 2 -> 2 0 1 0 -> No Page Fault 3 -> 2 3 1 0 -> 2 3 0 4 -> 4 3 0 2 -> 4 2 0 3 -> 4 2 3 0 -> 0 2 3 3 -> No Page Fault 2 -> No Page Fault 1 -> 0 1 3 2 -> 0 1 2 0 -> No Page Fault 1 -> No Page Fault 7 -> 7 1 2 0 -> 7 0 2 1 -> 7 0 1 Total page faults : 15.
```

EX NO: 11B — Page Replacement: Least Recently Used (LRU)

To write a C program to implement LRU page replacement algorithm.

PROGRAM CODE (LRU.C):

```
#include <stdio.h> int findLRU(int time[], int n) { int i, minimum = time[0], pos = 0; for(i = 1; i < n; ++i) { if(time[i] < minimum) { minimum = time[i]; pos = i; } } return pos; } int main() { int no_of_frames, no_of_pages, frames[10], pages[30], counter = 0, time[10]; int flag1, flag2, i, j, pos, faults = 0; printf("Enter number of frames: "); scanf("%d", &no_of_frames); printf("Enter number of pages: "); scanf("%d", &no_of_pages); printf("Enter reference string: "); for(i = 0; i < no_of_pages; ++i) scanf("%d", &pages[i]); for(i = 0; i < no_of_frames; ++i) frames[i] = -1; for(i = 0; i < no_of_pages; ++i) { flag1 = flag2 = 0; for(j = 0; j < no_of_frames; ++j) { if(frames[j] == pages[i]) { counter++; time[j] = counter; flag1 = flag2 = 1; break; } } if(flag1 == 0) { for(j = 0; j < no_of_frames; ++j) { if(frames[j] == -1) { counter++; faults++; frames[j] = pages[i]; time[j] = counter; flag2 = 1; break; } } } if(flag2 == 0) { pos = findLRU(time, no_of_frames); counter++; faults++; frames[pos] = pages[i]; time[pos] = counter; } printf("\n"); for(j = 0; j < no_of_frames; ++j) printf("%d\t", frames[j]); } printf("\n\nTotal Page Faults = %d\n", faults); return 0; }
```

OUTPUT:

```
Enter number of frames: 3 Enter number of pages: 6 Enter reference string: 5 7 5 6 7 3 5 -1 -1 5 7 -1 5 7 -1 5 7 6 5 7 6 3 7 6 Total Page Faults = 4
```

EX NO: 11C — Page Replacement: Optimal

To write a C program to implement Optimal page replacement algorithm.

PROGRAM CODE (OPTIMAL.C):

```
#include <stdio.h> int nof, nor, ref[50], frm[50], optcal[50]; int optvictim(int index) { int i, j, temp, notfound; for(i = 0; i < nof; i++) { notfound = 1; for(j = index; j < nor; j++) { if(frm[i] == ref[j]) { notfound = 0; optcal[i] = j; break; } } if(notfound == 1) return i; } temp = optcal[0]; for(i = 1; i < nof; i++) if(temp < optcal[i]) temp = optcal[i]; for(i = 0; i < nof; i++) if(frm[temp] == frm[i]) return i; return 0; } int main() { int i, j, flag, pf = 0, count = 0, victim = -1; printf("Enter the no. of frames: "); scanf("%d", &nof); printf("Enter the no. of reference string: "); scanf("%d", &nor); printf("Enter the reference string: "); for(i = 0; i < nor; i++) scanf("%d", &ref[i]); for(i = 0; i < nof; i++) frm[i] = -1; for(i = 0; i < nor; i++) { flag = 0; for(j = 0; j < nof; j++) { if(frm[j] == ref[i]) { flag = 1; break; } } if(flag == 0) { count++; if(count <= nof) victim++; else victim = optvictim(i); pf++; frm[victim] = ref[i]; } } printf("\nNumber of page faults: %d\n", pf); return 0; }
```

OUTPUT:

```
OPTIMAL PAGE REPLACEMENT ALGORITHM Enter the no. of frames: 3 Enter the no. of reference string: 8 Enter the reference string: 7 0 1 2 0 3 0 4 Number of page faults: 5
```

EX NO: 12 — File Organization Techniques (Single and Two-Level)

To implement Single Level Directory and Two-Level Directory File Organization Structures in C.

A. SINGLE LEVEL DIRECTORY STRUCTURE

```
#include <stdio.h> void main() { int count, i; char fname[10][20]; printf("Enter the number of files: "); scanf("%d", &c ount); for(i = 0; i < count; i++) { printf("Enter file %d name: ", i + 1); scanf("%s", fname[i]); } printf("\nRoot Directory File s:\n"); for(i = 0; i < count; i++) printf(" --> %s\n", fname[i]); }
```

B. TWO-LEVEL DIRECTORY STRUCTURE

```
#include <stdio.h> struct { char dname[10], fname[10][10]; int fcnt; } dir[10]; void main() { int dcnt, i, j; printf("Enter number of user directories: "); scanf("%d", &dcnt); for(i = 0; i < dcnt; i++) { printf("Enter directory %d name: ", i + 1); sca nf("%s", dir[i].dname); printf("Enter number of files for %s: ", dir[i].dname); scanf("%d", &dir[i].fcnt); for(j = 0; j < dir[i].fcn t; j++) { printf("Enter file %d name: ", j + 1); scanf("%s", dir[i].fname[j]); } } printf("\nDirectory Hierarchy:\n"); for(i = 0; i < dcnt; i++) { printf("User Directory: %s\n", dir[i].dname); for(j = 0; j < dir[i].fcnt; j++) printf("  └-- %s\n", dir[i].fname[j]); } }
```

OUTPUT EXECUTION:

```
Enter name of dir/file (under null): Hai How many users (for Hai): 1 Enter name of dir/file (under Hai): Hello How many files (for Hello): 1 Enter name of dir/file (under Hello): welcome Directory Hierarchy: User Directory: Hai  └-- Dire ctory: Hello  └-- File: welcome
```